

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
ASSESSMENT TOOL 2 – Industry Problem Solving Task

Course Code: CSA06	Course Title: Design and Analysis of Algorithms
CO Assessed: CO4: Articulation (BL4, BL5)	Assessment: Assessment Tool 2 – Industry Problem Solving Task
Weightage: 20% of CIA	Total Marks: 100

CO4 Statement: Solve optimization problems using dynamic programming and greedy strategies.

Student Name: DUDEKULA HUSSAINIAH	Reg. No.: 192425300
Section / Batch: _____	Date: _____

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given questions.
- Explain in detail with sample code if needed.
- Clearly identify all key issues.
- Provide a thorough analysis with validated results and performance evaluation.

Question Type	Focus Area	Questions
Industry Problem Solving Task	Focus on Solving optimization problems using dynamic programming and greedy strategies	Q10

SECTION A – Industry Problem Solving Task

Code of Conduct

I DUDEKULA HUSSAINIAH (Name / Reg No: 192425300) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: D.HUSSAINIAH

RUBRICS FOR CALCULATION

Industry Problem Solving Task

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
----------	-----	---------------------	----------------------	----------------------	---------------------

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%				
Application of Engineering Concepts	25%				
Solution Design	25%				
Use of Tools	15%				
Analysis	10%				
Professionalism	5%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name: _____	Name: _____	Name: _____
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Job Sequencing with Deadlines: A Greedy Algorithm for Profit Maximization in Digital Payment / Transaction Scheduling Systems

Student Name	DUDEKULA HUSSAINAIAH
Register Number	192425300
Question	Q10 – Job Sequencing – Payment / Profit Maximization
Algorithm Design Technique	Greedy Method
Industry Domain	Digital Payments / Fintech Transaction Processing

1. Aim

To study, design and analyse the Job Sequencing with Deadlines problem, apply a Greedy algorithm to solve it, and evaluate how the same approach maps onto a real industry problem — a digital payment system that must select and schedule payout/settlement jobs, each with a deadline and an associated profit, so that the total profit earned within the available time slots is maximum.

2. Industry Problem Statement

A company must schedule jobs with deadlines and associated profits such that maximum profit is earned within given time constraints. Analyze how the Job Sequencing with Deadlines problem can be solved using a Greedy approach. Design a solution by explaining the selection and scheduling strategy. Justify why the greedy choice leads to optimal profit. Suggest appropriate tools and data structures for implementation. Analyze time complexity and interpret results in terms of business profit optimization.

This experiment record applies the above theme to a concrete financial-industry setting — payout/settlement job scheduling in a digital payment system — since the profit-maximisation-under-deadline structure is identical in both framings.

3. Real-World Industry Context

Job Sequencing with Deadlines is not a textbook curiosity — versions of it run inside real production systems today:

- Payment gateways (Razorpay, Stripe, PayU) queue settlement/payout batches that must clear before a bank cut-off time; each batch carries a transaction-fee value, and only a limited number of batches can be pushed before the deadline.
- UPI / NEFT / RTGS back-office systems prioritise high-value transfers when the settlement window is about to close, so that maximum transaction value clears before the window shuts.
- Cloud billing and ad-auction systems (Google Ads, AWS spot billing) select which billable jobs to execute before a rate-card window expires, to maximise revenue per cycle.
- Freelance/gig marketplaces (Upwork, Swiggy Genie) assign the highest-paying task to a worker's last available slot before its due time.

In every case the shape of the problem is identical: a set of jobs (or transactions), each with a deadline and a profit (or fee), one machine/slot processes one unit of work per time unit, and the goal is to pick a subset and an order that keeps every chosen job inside its deadline while maximising total profit.

4. Problem Analysis

Formally: given n jobs $J = \{J_1, J_2, \dots, J_n\}$, where each job J_i has a deadline d_i and a profit p_i , and each job takes exactly 1 unit of time on a single available processor, find a schedule (a subset and an assignment of jobs to slots $1 \dots \max(d_i)$) that:

- Executes at most one job per time slot,
- Executes every scheduled job strictly on or before its deadline, and
- Maximises the sum of profits of the scheduled jobs.

This is a selection + ordering problem under a single resource constraint (one payment-processing slot per unit time) — structurally the same shape as the 0/1 Knapsack and Activity Selection problems studied elsewhere in this course, but here the 'capacity' is the number of available time slots rather than weight.

5. Why a Greedy Approach Works Here

The Greedy method is provably optimal for this problem because it has the Greedy-choice property and Optimal Substructure:

- Greedy-choice property: always considering the highest-profit job first and placing it into the latest free slot at or before its deadline never blocks a better solution — a later, lower-profit job can only ever compete for the same or an earlier slot, so giving priority to profit first cannot lose total value.
- Optimal substructure: once the highest-profit job is placed, the remaining problem ($n - 1$ jobs, one slot removed) is an identical, smaller instance of Job Sequencing with Deadlines — so an optimal solution to the whole problem is built from optimal solutions to the sub-problem.

Because both properties hold, sorting once by profit and greedily placing each job as late as possible before its deadline is guaranteed to reach the same optimum a brute-force search over all subsets would — without the exponential cost of trying every combination.

6. Algorithm Design

Step-by-step design:

- Step 1 — Sort all n jobs in non-increasing order of profit.
- Step 2 — Find d , the maximum deadline among all jobs; create d time slots, all initially free.
- Step 3 — For each job in the sorted order (highest profit first): search for a free slot at time t , where t ranges from $\min(d_i, d)$ down to 1.
- Step 4 — If a free slot is found, assign the job to that slot and add its profit to the running total. If no free slot exists on or before its deadline, discard the job.
- Step 5 — Repeat Step 3–4 for all n jobs. The set of filled slots is the final schedule; the running total is the maximum achievable profit.

7. Pseudocode

```

JOB-SEQUENCING(jobs[1..n])
  sort jobs in decreasing order of profit
  d <- max(deadline[i]) for all i
  slot[1..d] <- FREE
  totalProfit <- 0
  for i <- 1 to n:
    t <- min(deadline[i], d)
    while t >= 1 and slot[t] != FREE:
      t <- t - 1
    if t >= 1:
      slot[t] <- jobs[i]
      totalProfit <- totalProfit + profit[i]
  return slot[], totalProfit

```

8. Worked Example

Consider four payout jobs competing for daily settlement slots:

Job	Profit (₹)	Deadline (slot)
J3	40	1
J4	30	1
J1	20	4
J2	10	1

Sorted by profit (descending): J3(₹40, d=1) → J4(₹30, d=1) → J1(₹20, d=4) → J2(₹10, d=1). Placing them greedily into the latest free slot at or before each deadline gives:

Time Slot	Outcome	Reason
-----------	---------	--------

Time Slot	Outcome	Reason
Slot 1 (day 1)	J3 assigned	Highest-profit job whose deadline permits day 1
Slot 2 (day 2)	Empty	No pending job has deadline ≥ 2 left unplaced at this step
Slot 3 (day 3)	Empty	—
Slot 4 (day 4)	J1 assigned	J1 deadline = 4, latest free slot ≤ 4 is slot 4

Result: jobs J3 and J1 are scheduled; J4 and J2 are dropped because no free slot remains within their deadline. Maximum achievable profit = ₹40 + ₹20 = ₹60, using only 2 of the 4 available slots — the same figure a full $2^4 = 16$ -subset brute-force search would return, reached here in a single sorted pass.

9. Why Greedy Was Chosen Over Dynamic Programming Here

It is worth checking why this problem does not need the heavier Dynamic Programming machinery used elsewhere in this course (e.g. Knapsack, Coin Change):

- DP is needed when a problem lacks the greedy-choice property — i.e. the locally best choice can block a better global solution, forcing you to explore multiple choices and remember overlapping sub-results in a table.
- Job Sequencing with Deadlines does have the greedy-choice property (proved in Section 5): taking the highest-profit job first and placing it as late as possible never blocks a better arrangement of the remaining jobs, because every later job's feasible slot range is a subset of an earlier slot range it could have used.
- Consequently a single sort plus one pass of slot placement reaches the same optimum a DP table or brute-force subset search would — at lower time and space cost.
- Practical rule of thumb used in industry: reach for Greedy first because it is simpler and faster; fall back to DP only when a counter-example shows the greedy-choice property fails for the specific problem variant (for instance, when jobs can take more than one time unit, simple greedy slot-filling no longer guarantees optimality and a DP or ILP formulation is required).

10. Tools and Platforms Used in Practice

- Languages: Java / C++ for latency-critical settlement engines; Python for prototyping and back-office batch jobs.
- Data structures: a boolean/array slot table for small n ; a Disjoint Set Union (Union-Find with path compression) structure for large n , since it turns each 'find nearest free slot' query into near $O(1)$ time.
- Scheduling frameworks: Quartz Scheduler / Kubernetes CronJobs to trigger batch settlement windows; Apache Kafka / RabbitMQ to queue incoming payout jobs before the greedy scheduler consumes them; Redis for fast in-memory slot-availability lookups.
- Observability: Prometheus/Grafana dashboards to track scheduled-vs-dropped job counts and realised profit per settlement cycle in production.

11. Complexity Analysis

Step	Time Complexity	Space Complexity
Sort transactions/jobs by profit	$O(n \log n)$	$O(1)$ (in-place sort)
Slot search — linear scan (array of booleans)	$O(n \times d)$	$O(d)$
Slot search — Disjoint Set Union (Union-Find)	$O(n \alpha(n)) \approx O(n)$	$O(d)$
Overall (array method)	$O(n^2)$ worst case	$O(n + d)$
Overall (DSU-optimized method)	$O(n \log n)$	$O(n + d)$

With the plain array-based slot search, worst-case time is $O(n^2)$ when deadlines cluster near the maximum, since each job may scan back through most of the slot array. Replacing the slot array with a Disjoint-Set-Union 'find nearest free slot $\leq t$ ' structure reduces this to effectively $O(n \log n)$, dominated by the initial sort — the version used in large-scale payment back-ends handling millions of transactions.

12. Interpretation of Results in an Industry Context

For a financial system, the output of this algorithm is directly a business decision: which transactions clear this cycle and which are deferred or rejected. The greedy strategy guarantees the maximum possible fee/profit revenue extractable from a fixed settlement window, which is exactly the metric a payments company optimises for. Because the algorithm runs in near-linear time after sorting, it scales to the transaction volumes seen in real UPI/card-network settlement cycles (tens of millions of transactions per window) without becoming the bottleneck in the pipeline.

13. Conclusion

The Job Sequencing with Deadlines problem is a clean example of how a Greedy algorithm, backed by the greedy-choice property and optimal substructure, solves a real financial-scheduling problem optimally and efficiently. The same slot-based greedy pattern — sort by value, place into the latest feasible slot — recurs across payment settlement, ad-serving, and gig-economy task assignment, which is why it is a standard building block in industry scheduling engines rather than a purely academic exercise.